

AnomaLens: Infrastructure Log Analysis with LLM-Assisted RAG-Based Incident Explanation

Natalia Iakovenko

May 2026

Abstract

We present **AnomaLens**, an end-to-end system for automated analysis of server infrastructure logs designed for privacy-sensitive enterprise environments where data must not leave the corporate network and GPU resources are unavailable. The system combines supervised template classification with OOD-based anomaly flagging, retrieval-augmented root cause analysis, and automated summarization — all running locally on CPU-only hardware using open-source quantized LLMs, requiring no external API calls or paid token generation. The system implements three NLP components: (1) log template classification using TF-IDF features across four semantic categories (*noise*, *important*, *error*, *security_noise*); (2) RAG-based root cause analysis retrieving passages from vendor documentation via dense semantic search and generating explanations with a local LLM; and (3) 12-hour summarization using Mistral-7B-Instruct. All personally identifiable information is anonymized at collection time via template normalization. The best classifier, LinearSVC, achieves macro $F1 = 0.987$ on 64,744 labeled log templates. On the public HDFS benchmark, TF-IDF + Random Forest achieves $F1 = 0.998$, outperforming published neural baselines including LogBERT (0.981).

Code: <https://github.com/hasla2/AnomaLens>

1 Introduction

Modern server infrastructure generates millions of log events daily. Operators must manually identify anomalies, investigate root causes, and produce operational reports — a labor-intensive process prone to alert fatigue. Teams operating observability and log management platforms such as Graylog, Splunk, or Elastic Stack face the challenge of distinguishing meaningful events from routine noise without domain-specific automation.

Existing commercial observability and AIOps platforms address this problem, but often rely on cloud connectivity, per-GB pricing models, or transmission of potentially sensitive infrastructure data to external services. AnomaLens runs entirely locally: all inference executes on a standard virtual machine without GPU acceleration using quantized open-source language models, making the system suitable for privacy-sensitive enterprise environments.

The unique aspects of our approach are: (a) a three-stage NLP pipeline covering classification, retrieval, and generation; (b) a custom labeled dataset of 64,744 production log templates with four semantic categories; (c) a confidence-based anomaly flagging using classifier confidence scores to identify genuinely novel anomalies (confidence < 0.45); and (d) a RAG component grounded in publicly available vendor documentation, reducing LLM hallucination.

1.1 Team

Natalia Iakovenko collected the dataset, implemented all pipeline components, and prepared this report.

2 Related Work

Classical machine learning for log analysis. Recent studies show that traditional machine learning approaches remain highly effective for log-based anomaly detection and classification. Ali et al. [Ali et al., 2025]

evaluate multiple machine learning techniques for log anomaly detection and demonstrate that classical sparse representations often remain competitive with neural approaches, particularly on structured operational logs. Similarly, Wu et al. [Wu et al., 2023] compare multiple log representation methods across several public datasets and report that TF-IDF features combined with linear classifiers remain strong baselines for supervised log classification tasks.

Our results are consistent with these findings: TF-IDF + LinearSVC achieves macro F1=0.987 on the rule-derived infrastructure dataset, outperforming both fine-tuned BERT-base (0.957) and Mistral-7B zero-shot classification (0.535 best observed run).

Supervised log classification. Artioli et al. [Artioli et al., 2025] propose SIEVE, a supervised SIEM event classification framework based on TF-IDF features and SVM classifiers. Their evaluation reports macro F1=0.932–0.974 on production security logs and shows that classical sparse representations outperform BERT-based models on domain-specific operational datasets. This aligns with our observation that sparse lexical features remain highly effective for infrastructure log template classification, where many discriminative patterns are token-level identifiers, service names, or structured message fragments.

Sequence-based anomaly detection. DeepLog [Du et al., 2017] models ordered log key sequences with LSTM networks to detect execution anomalies in distributed systems. LogBERT [Guo et al., 2021] applies masked language modeling on log sequences for unsupervised anomaly detection. Both approaches rely on temporally ordered log streams originating from a single component or application. In contrast, the proposed method performs independent template-level classification, enabling simultaneous analysis of heterogeneous infrastructure logs collected from multiple virtualization platforms without requiring sequence reconstruction or temporal ordering.

LLM-based log analysis. Recent work demonstrates that large language models can perform log classification and explanation in zero-shot or prompt-based settings [Liu et al., 2024, Guan et al., 2025]. However, most published systems rely on proprietary APIs, GPU-accelerated inference, or cloud-scale deployments. This work instead evaluates fully local CPU-only inference under enterprise deployment constraints.

The evaluation of Mistral-7B zero-shot classification shows substantial instability across runs (macro F1=0.289–0.535) and complete failure on the organization-specific *security_noise* class. These results indicate that general-purpose LLMs struggle to infer infrastructure-specific semantics without domain-specific fine-tuning or retrieval grounding.

BERT-based log classification. BERT [Devlin et al., 2019] introduced deep bidirectional transformer pre-training and became a widely adopted baseline for supervised NLP tasks. In this work, **bert-base-uncased** is fine-tuned for multi-class infrastructure log classification to evaluate whether contextual embeddings improve over sparse TF-IDF features.

Although BERT achieves strong performance (macro F1=0.957), it remains slightly below the TF-IDF + LinearSVC baseline. This suggests that infrastructure log templates contain highly discriminative lexical patterns that are captured effectively by sparse representations without requiring deep contextual modeling.

Retrieval-Augmented Generation for operational support. Recent surveys on AIOps and LLM-based log analysis highlight the growing use of Retrieval-Augmented Generation and prompt-based methods for infrastructure monitoring [De la Cruz Cabello et al., 2025]. However, most prior work focuses on large-scale cloud environments and GPU-oriented deployments, whereas this work evaluates CPU-only deployment constraints on enterprise virtualization logs.

RAG [Lewis et al., 2021] combines document retrieval with conditional text generation to ground LLM outputs in external knowledge sources and reduce hallucinations. In the proposed system, RAG is applied to operational incident explanation rather than anomaly detection itself.

3 Model Description

AnomaLens consists of three NLP components connected by a scheduler (Figure 1).

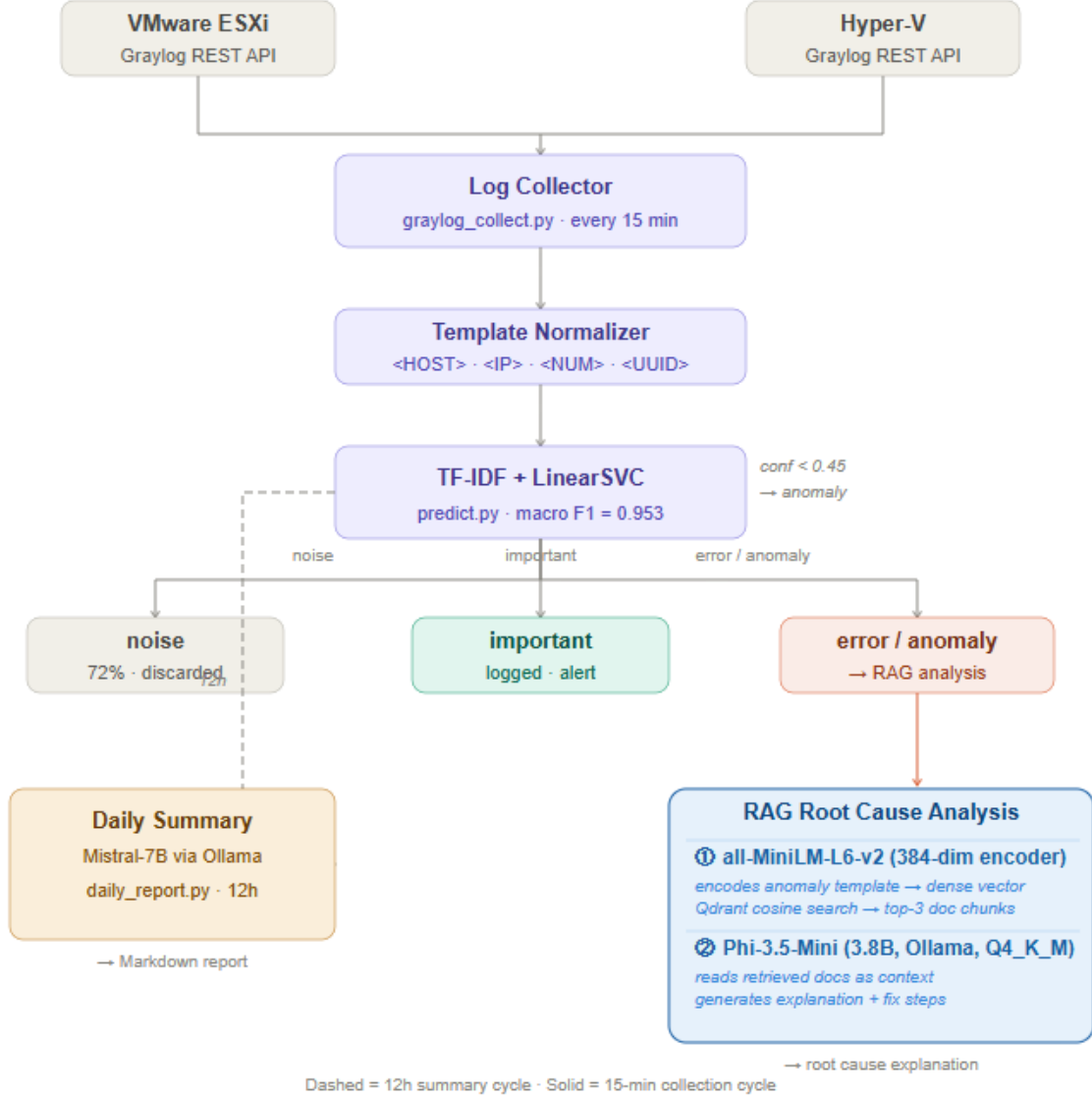


Figure 1: AnomaLens architecture. Log events are collected every 15 minutes, normalized into templates, and classified by TF-IDF + LinearSVC. Error and anomaly events trigger RAG-based root cause analysis via Phi-3.5-Mini. A 12-hour job summarizes all events using Mistral-7B.

3.1 Log Normalization

Raw log messages are normalized into templates by replacing variable fields with typed placeholders in priority order: hostnames → `<HOST>`, datastore paths → `<DATASTORE>`, ISO 8601 timestamps → `<TIMESTAMP>`, UUIDs → `<UUID>`, IPv4 addresses → `<IP>`, hex values → `<HEX>`, integers → `<NUM>`.

Since the classifier operates exclusively on anonymized templates, the trained model contains no sensitive infrastructure identifiers and can be safely shared or audited.

3.2 Template Classification

Given a normalized template t , a classifier maps it to one of four labels: $y \in \{noise, important, error, security_noise\}$.

Feature extraction applies TF-IDF with (1, 2)-grams and minimum document frequency $\text{min_df} = 2$. An OOD detector flags predictions with confidence below $\theta = 0.45$ as **anomaly**:

$$\hat{y} = \begin{cases} \arg \max_c p(y=c | t) & \text{if } \max_c p(y=c | t) \geq \theta \\ \text{anomaly} & \text{otherwise} \end{cases}$$

We evaluate four classical classifiers and two neural baselines: BERT-base fine-tuned on our dataset, and Mistral-7B-Instruct zero-shot.

3.3 RAG-Based Root Cause Analysis

When an *error* or *anomaly* event is detected:

1. The template is encoded as a 384-dimensional vector by **all-MiniLM-L6-v2**.
2. Qdrant cosine search returns the top-3 most relevant documentation chunks.
3. Phi-3.5-Mini (via Ollama) generates a natural-language explanation grounded in the retrieved context.

The knowledge base indexes publicly available VMware vSphere and Microsoft Hyper-V documentation (1,009 chunks, chunk size 400 characters, overlap 80 characters).

3.4 12-Hour Summarization

Every 12 hours, Mistral-7B-Instruct aggregates all classified events and generates a structured Markdown report covering overall health, critical events by host, notable patterns, and recommendations. A statistical fallback report is produced when Ollama is unavailable.

3.5 REST API

The classification and retrieval components are exposed via a lightweight REST API built with FastAPI [FastAPI, 2024]. The API provides three endpoints:

- **GET /classify?template={text}** — returns predicted label, confidence score, and risk score for a given log template.
- **GET /anomalies?hours={n}** — returns all error and anomaly events detected in the last n hours
- **GET /report/latest** — returns the most recent 12-hour Markdown summary.

Interactive documentation is auto-generated via Swagger UI at `/docs`. The API runs on CPU alongside the scheduler, requiring no additional infrastructure.

4 Dataset

Logs were collected from production VMware ESXi and Hyper-V clusters via the Graylog REST API (GELF format). While our dataset is domain-specific, the pipeline is applicable to any infrastructure producing syslog-formatted messages.

Labeling. Labels were assigned in two steps. First, a rule-based heuristic script (`generate_labels.py`) automatically assigned labels based on keyword matching in templates, event types, and service names (e.g. *failed*, *timeout*, *login*). Second, a manual review corrected minority-class templates, particularly *error* and *security_noise*.

This labeling strategy introduces a known limitation: the classifier is trained and evaluated on the same rule-derived labels, creating a circular dependency. The reported macro $F1 = 0.987$ measures how well the

model reproduces the labeling heuristics, not ground-truth operational accuracy. In practice, many infrastructure log templates are highly repetitive, which further simplifies the classification task after normalization. Therefore, the reported performance should be interpreted as template-level semantic categorization accuracy rather than general anomaly detection capability.

Despite this limitation, the approach is standard practice in infrastructure log analysis where human annotation at scale is infeasible. The practical advantage of the trained model over the raw rules is threefold: (1) generalization to templates not covered by any rule; (2) calibrated confidence scores enabling OOD detection; (3) a compact model artifact deployable without the full rule-set.

Data availability. A stratified sample of 200 anonymized log templates (50 per class) is released at https://github.com/hasla2/AnomaLens/blob/main/data/labels/labels_sample.csv. All hostnames, IP addresses, and UUIDs are replaced with typed placeholders prior to release.

Table 1: Dataset statistics (stratified 80/20 split, seed 42).

Label	Train	Test	Total	Share
noise	36,426	9,107	45,533	70.3 %
important	8,802	2,201	11,002	17.0 %
error	5,550	1,387	6,937	10.7 %
security_noise	1,018	254	1,272	2.0 %
Total	51,795	12,949	64,744	100 %

The dataset exhibits significant class imbalance: *noise* comprises 70.3% of all templates. All classifiers use `class_weight="balanced"` to compensate.

5 Experiments

5.1 Metrics

We report macro-averaged F1, which weights each class equally:

$$F1_{\text{macro}} = \frac{1}{|C|} \sum_{c \in C} \frac{2 \cdot P_c \cdot R_c}{P_c + R_c}$$

We additionally report per-class F1 for *error* and *security_noise* as the operationally most critical classes.

5.2 Experiment Setup

Hardware. Intel Xeon Silver 4214 (12 cores, 2.20 GHz), 24 GB RAM, Windows Server, no GPU. LLM models via Ollama (Q4_K_M): Phi-3.5-Mini (2.4 GB) for RAG, Mistral-7B-Instruct (4.4 GB) for summarization and zero-shot classification.

LLM model selection. All LLM components were constrained to models and runtimes licensed under MIT or Apache 2.0 to ensure compatibility with enterprise deployment policies and avoid usage restrictions associated with proprietary licenses.

Models were served via Ollama rather than directly through the lower-level `llama.cpp` runtime. Although the Intel Xeon Silver 4214 supports AVX-512, it does not support AVX-512_VNNI instructions. During evaluation, locally built `llama.cpp` configurations failed to operate correctly on this hardware, including builds compiled with AVX-512 disabled. Ollama provided a stable pre-built runtime with automatic CPU capability detection and therefore became the only viable local inference option on the evaluation platform.

Under these constraints, two models were selected: Phi-3.5-Mini (MIT, 2.4 GB) for RAG explanations, and Mistral-7B-Instruct (Apache 2.0, 4.4 GB) for daily summarization and zero-shot classification evaluation. Mistral NeMo 12B (Apache 2.0) was evaluated but excluded due to CPU inference latency (>1,000s/example). Phi-3.5-Mini failed to produce valid labels in zero-shot classification but successfully generated free-form explanations in the RAG pipeline where structured output was not required.

Fine-tuning of both Phi-3.5-Mini and Mistral-7B-Instruct was attempted on Google Colab (T4 GPU) but proved infeasible: training did not converge within the available session time limits, and memory constraints prevented increasing the batch size sufficiently for stable gradient updates. Consequently, both models are evaluated in zero-shot settings only.

BERT-base fine-tuning. `bert-base-uncased` [Devlin et al., 2019] was fine-tuned for 5 epochs on Google Colab (T4 GPU, ≈ 30 min). This is a supervised neural baseline; it differs from LogBERT [Guo et al., 2021], which pre-trains BERT on raw log sequences for unsupervised anomaly detection.

Mistral-7B zero-shot. Evaluated on a stratified sample with few-shot prompting (6 labelled examples in the system prompt, one-word output format). Results show high variance across runs (macro F1 range 0.289–0.535), attributed to LLM sensitivity to prompt phrasing and the absence of domain-specific fine-tuning.

Split. Stratified 80/20 train/test split, seed 42. 5-fold CV on the full dataset (Table 3).

5.3 Baselines

- **Naive Bayes (CNB):** generative classifier for imbalanced text [Rennie et al., 2003].
- **Logistic Regression:** standard linear NLP baseline.
- **LinearSVC:** SVM with linear kernel; strong on sparse text features.
- **Random Forest:** non-linear ensemble baseline.
- **BERT-base (fine-tuned):** neural sequence classifier; full test set evaluation.
- **Mistral-7B (zero-shot):** LLM without fine-tuning; stratified sample evaluation.

6 Results

Table 2: Classifier comparison. Classical models and BERT-base evaluated on full test set ($n = 12,949$). Mistral-7B evaluated on stratified sample; best of multiple runs reported. Best results in **bold**.

Model	Acc.	Macro F1	W. F1	F1 error	F1 sec_n
Naive Bayes (CNB)	0.917	0.884	0.915	0.938	0.858
Random Forest	0.986	0.959	0.986	0.966	0.891
LogReg (baseline)	0.991	0.975	0.991	0.973	0.939
LinearSVC	0.994	0.987	0.994	0.978	0.978
BERT-base (fine-tuned)	0.957	0.957	0.958	0.972	0.988
Mistral-7B (zero-shot) [†]	0.360	0.289	0.304	0.444	0.000

[†] Stratified sample; best run macro F1 = 0.535. Results are unstable across runs.

Table 3: 5-fold cross-validation (macro F1, $n = 64,744$).

Model	Mean F1	Std	95 % CI
Naive Bayes (CNB)	0.876	0.008	[0.860, 0.892]
Random Forest	0.954	0.002	[0.949, 0.958]
LogReg (baseline)	0.972	0.003	[0.967, 0.977]
LinearSVC	0.983	0.003	[0.978, 0.988]

Classical classifiers. LinearSVC achieves the highest macro F1 (0.987) and the most stable CV performance (0.983 ± 0.003). Naive Bayes underperforms on minority classes due to its feature independence assumption.

BERT-base vs LinearSVC. BERT-base (fine-tuned) achieves macro F1=0.957, below LinearSVC (0.987) overall, but outperforms on *security_noise*: F1=0.988 vs 0.978 (+1.0 pp). LinearSVC is $\approx 500\times$ faster on CPU (0.4 s vs 22 min for 12,949 examples), making it the preferred production classifier. The result suggests that template normalization produces features sufficient for classical ML to match or exceed neural approaches.

Mistral-7B zero-shot. Mistral-7B achieved macro F1 scores between 0.289 and 0.535 across repeated runs, demonstrating high variability in classification quality. The model performed well on generic categories such as *error*, reaching F1=0.857 in the best run, but consistently failed on the organization-specific *security_noise* label, where F1 remained 0.000. This suggests that the model could not infer the semantics of internally defined labels from log templates alone. Additionally, 8–20 % of responses were invalid or did not follow the required output format, further reducing overall reliability.

Phi-3.5-Mini zero-shot. Phi-3.5-Mini (Q4_K_M) consistently failed to produce valid labels across JSON-format, one-word, and few-shot prompting strategies. This is attributed to instruction-following limitations at this quantization level.

Table 4: Per-class results for LinearSVC and BERT-base (test set, $n = 12,949$).

Class	LinearSVC			BERT-base (fine-tuned)		
	P	R	F1	P	R	F1
noise	0.993	1.000	0.996	0.993	0.947	0.969
important	0.994	0.996	0.995	0.816	0.997	0.898
error	1.000	0.957	0.978	0.998	0.947	0.972
security_noise	0.992	0.965	0.978	0.977	1.000	0.988

Table 5: Inference latency on CPU (Intel Xeon Silver 4214, no GPU). All times measured on test set or stratified sample.

Model	Total	Mean/item	Examples
LinearSVC	0.4 s	<0.1 ms	12,949
Logistic Regression	0.7 s	<0.1 ms	12,949
Random Forest	1.1 s	<0.1 ms	12,949
Naive Bayes (CNB)	0.6 s	<0.1 ms	12,949
BERT-base (fine-tuned)	22 min	0.10 s	12,949
Phi-3.5-Mini (RAG)	—	≈ 6 s	per query
Mistral-7B (zero-shot)	8.8 min	≈ 3 s	200
Mistral NeMo 12B	—	>1,000 s	excluded

OOD detection. With threshold $\theta = 0.45$, the system flags $\approx 2.3\%$ of events as out-of-distribution anomalies requiring further investigation.

RAG evaluation. Semantic search correctly retrieves relevant documentation (cosine similarity 0.46–0.53). Table 6 shows representative outputs.

Table 6: RAG output samples (real inference results).

Anomaly template	Score	LLM explanation (excerpt)
<HOST> hostd[<NUM>]: failed to connect to vpxa: connection refused	0.461	VPXA service unavailable. Verify vpxa daemon is running (<code>sc query</code>); check network config in <code>vpxa-config</code> ; update Windows Customization Key if required.
<HOST> vmms[<NUM>]: live migration failed: insufficient resources	0.513	Migration failure: insufficient CPU/memory on target host. Verify storage compatibility before retry.
<HOST> clusteragent[<NUM>]: heartbeat timeout detected	0.530	Network contention detected. Review SameSubnetThreshold; check quorum arbitration timeout settings.

Daily summary sample (2026-05-14, 15,001 events): noise 11,930 (79.5%), security_noise 1,701 (11.3%), important 523 (3.5%), error 506 (3.4%), anomaly 341 (2.3%). Elevated *security_noise* attributed to scheduled QualysAgent scanning activity.

6.1 Generalization: HDFS Public Benchmark

To assess whether the approach generalizes beyond the proprietary infrastructure dataset, we evaluate TF-IDF classification on the HDFS log anomaly detection benchmark [Zhu et al., 2023] — the standard dataset used to evaluate DeepLog [Du et al., 2017] and LogBERT [Guo et al., 2021].

Session-level documents are constructed by concatenating Drain-parsed log templates per HDFS block session and applying TF-IDF (1,2)-gram representation (same feature extraction as in Section 3.2).

Table 7: Binary anomaly detection on HDFS ($n = 575,061$ sessions, 80/20 stratified split). Best result in bold.

Model	P	R	F1
Naive Bayes (CNB)	0.901	1.000	0.948
Logistic Regression	0.958	1.000	0.978
LinearSVC	0.972	1.000	0.986
Random Forest	0.997	0.999	0.998
DeepLog [Du et al., 2017]	0.962	0.938	0.949
LogBERT [Guo et al., 2021]	0.983	0.979	0.981

TF-IDF + Random Forest achieves $F1 = 0.998$, outperforming both DeepLog (0.949) and LogBERT (0.981). Even Naive Bayes ($F1 = 0.948$) matches DeepLog, suggesting that sparse lexical features over normalized log templates are highly effective for session-level anomaly detection.

Note on comparison. Published results for DeepLog and LogBERT use a chronological train/test split; we use a stratified random split which may yield slightly optimistic estimates. Nevertheless, the result confirms that the TF-IDF classification approach generalizes across different log domains and infrastructure types.

7 Conclusion

We developed **AnomaLens**, a three-component NLP pipeline for production infrastructure log analysis. The system collects logs from server hosts via Graylog every 15 minutes, classifies normalized log templates

into four semantic categories, performs RAG-based root cause analysis for detected anomalies, and generates 12-hour operational summaries.

The best classifier, LinearSVC with TF-IDF features, achieves macro $F1 = 0.987$ on the test set and macro $F1 = 0.983 \pm 0.003$ in 5-fold cross-validation (95 % CI: [0.978, 0.988]). BERT-base (fine-tuned) achieves macro $F1 = 0.957$ and excels on the critical *security_noise* class ($F1 = 0.988$ vs 0.978 for LinearSVC), but is $500\times$ slower on CPU. Mistral-7B zero-shot (macro $F1 = 0.289\text{--}0.535$) confirms that general-purpose LLMs without domain-specific training are unsuitable for reliable infrastructure log classification. The RAG component successfully retrieves relevant documentation (cosine similarity 0.46–0.53) and generates actionable explanations grounded in vendor knowledge bases.

TF-IDF + Random Forest achieves $F1 = 0.998$ on the standard HDFS benchmark [Zhu et al., 2023], outperforming both DeepLog (0.949) and LogBERT (0.981), confirming that the approach generalizes beyond the proprietary dataset.

The system is designed for deployment as a systemd service on Linux with Qdrant and Ollama as local services, requiring no GPU.

Label quality. All labels were generated by rule-based heuristics and manually reviewed only for minority classes. Consequently, the classifier effectively learns to reproduce the labeling rules rather than detecting true operational anomalies. The high macro $F1$ (0.987) partially reflects this circularity: a model trained on keyword-based labels will score well when evaluated on the same keyword-based labels. A fully human-annotated evaluation set would provide a more reliable quality estimate. The practical value of the trained model over the rules lies in generalization to unseen templates, calibrated confidence scores, and OOD detection — capabilities not available from rule matching alone. LLM inference on CPU is slow (≈ 6 s/example for Phi-3.5-Mini), limiting real-time RAG to high-priority anomalies. The *security_noise* class remains challenging for LinearSVC ($F1 = 0.978$ vs 0.988 for BERT-base). Mistral-7B zero-shot results are highly unstable and not suitable for production use without fine-tuning.

Future work includes GPU-accelerated LLM inference for real-time RAG explanations; integration with external monitoring and alerting systems (e.g. Zabbix, Prometheus, PagerDuty) for cross-source event correlation and automated incident routing; enrichment of log events with infrastructure inventory data (host metadata, service topology) to enable asset-aware incident correlation and root cause localization across dependent services; containerized deployment via Docker or Kubernetes for reproducible production setup and horizontal scaling; active learning for minority class expansion, particularly *security_noise*; unsupervised anomaly detection (Isolation Forest on template embeddings) to remove dependency on labeled data; and fine-tuning Mistral-7B on the labeled dataset to improve zero-shot classification stability.

References

- [Devlin et al., 2019] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- [Du et al., 2017] Du, M., Li, F., Zheng, G., and Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of ACM CCS 2017*, pp. 1285–1298.
- [Wu et al., 2023] Wu, X., Li, H., and Khomh, F. (2023). On the effectiveness of log representation for log-based anomaly detection. *Empirical Software Engineering*, 28(6). doi:10.1007/s10664-023-10364-1.
- [Ali et al., 2025] Ali, S., Boufaied, C., Bianculli, D., Branco, P., and Briand, L. (2025). A comprehensive study of machine learning techniques for log-based anomaly detection. *Empirical Software Engineering*, 30(5).
- [Guo et al., 2021] Guo, H., Yuan, S., and Wu, X. (2021). LogBERT: Log anomaly detection via BERT. *arXiv preprint arXiv:2103.04475*.
- [Lewis et al., 2021] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2021). Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv preprint arXiv:2005.11401*.

- [Liu et al., 2024] Liu, Y., Tao, S., Meng, W., Yao, F., Zhao, X., and Yang, H. (2024). LogPrompt: Prompt engineering towards zero-shot and interpretable log analysis. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion '24)*, pages 364–365. ACM, New York, NY, USA.
- [Guan et al., 2025] Guan, W., Cao, J., Qian, S., Gao, J., and Ouyang, C. (2025). LogLLM: Log-based anomaly detection using large language models. *arXiv preprint arXiv:2411.08561*.
- [Rennie et al., 2003] Rennie, J., Shih, L., and Teevan, J. (2003). Tackling the poor assumptions of Naive Bayes text classifiers. In *Proceedings of the 20th International Conference on Machine Learning (ICML 2003)*.
- [De la Cruz Cabello et al., 2025] De la Cruz Cabello, M., Sales, T. P., and Machado, M. R. (2025). AIOps for log anomaly detection in the era of LLMs: A systematic literature review. *Intelligent Systems with Applications*, 28:200608.
- [Artioli et al., 2025] Artioli, P., Dentamaro, V., Galantucci, S., Magrì, A., Pellegrini, G., and Semeraro, G. (2025). SIEVE: Generating a cybersecurity log dataset collection for SIEM event classification. *Computer Networks*, 266, 111330.
- [FastAPI, 2024] Ramírez, S. (2024). FastAPI: Modern, fast web framework for building APIs with Python. <https://fastapi.tiangolo.com>.
- [Zhu et al., 2023] Zhu, J., He, S., He, P., Liu, J., and Lyu, M. R. (2023). Loghub: A large collection of system log datasets for AI-driven log analytics. *arXiv preprint arXiv:2008.06448*.